

Suggested Medium titles (pick one)

1) Stop Copying Rows: The Simple Trick Behind Fast Databases (SV vs Bitmaps) 2) How Databases Go Fast: A “Keep/Ignore Note” That Saves Tons of Work 3) Selection Vectors vs Bitmaps — Explained Like You’re New to IT

Stop Copying Rows: The Simple Trick Behind Fast Databases (SV vs Bitmaps)

If you’re new to IT, databases can sound like magic.

Here’s the non-magic version:

Fast databases often avoid copying data. Instead, they keep data in place and carry a tiny “keep/ignore note” through the pipeline.

That “keep/ignore note” is what the research paper “**Filter Representation in Vectorized Query Execution**” (Ngom et al., 2021) is about.

This article explains the idea with simple examples, diagrams, and a small cheat-sheet.

The one-sentence meaning

A **filter representation** is just a way to remember:

“In this small group of rows, which ones should I keep using, and which ones should I ignore?”

Quick definitions (easy)

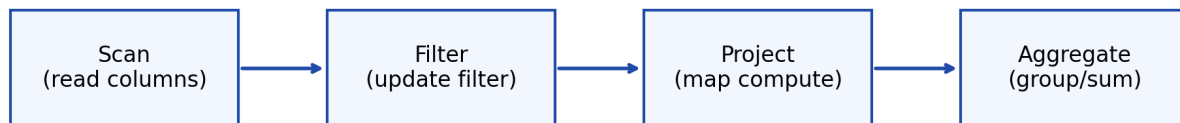
- **Row**: one record (example: one order).
- **Batch**: a small group of rows processed together (example: 1,000 rows).
- **Pipeline**: multiple steps in a row (scan → filter → compute → sum).
- **Filter**: a rule that keeps some rows and rejects others.
- **Filter representation**: the “keep/ignore note” attached to the batch.

The big picture (architecture)

A vectorized database processes **batches** through operators (think: stations on a factory line).

Diagram: batch + filter flows through the pipeline

Vectorized execution: operators pass a batch + a filter through the pipeline



Each operator processes a BATCH (vector) + carries a FILTER (SV or BM) so it knows which positions are still valid without copying rows.

Important idea:

- The batch values (columns) can stay in memory.
- The “keep/ignore note” changes as you apply filters.

Why does this exist? (the purpose)

When you filter data, you have two choices.

Option A: Copy only the survivors

You build a new output batch containing only the rows that passed.

This is simple, but copying again and again is expensive.

Option B: Don't copy. Keep the batch and carry a keep/ignore note

You keep the values in place. You only update the note that says which positions are valid.

That note is the filter representation.

Diagram: copy vs mask

Purpose: avoid copying data by tracking validity separately

Option A: Copy survivors (more memory work)

Input vector (8 rows) -> Output vector (only survivors)
You physically move/copy values for rows that pass the filter.

Option B: Keep vector, just mark validity (filter representation)

Input vector stays the same in memory.
You only update SV/BM to say which positions are valid.
Later operators read only valid positions (via SV) or mask them (via BM).

A tiny example with real numbers (8 rows)

We have one batch with 8 rows.

Positions: 0 1 2 3 4 5 6 7

amount: 20 150 5 130 200 10 90 180

Filter rule:

Keep rows where amount > 100

Survivors are positions: **1, 3, 4, 7**

Diagram: what changes after the filter

Example query step: WHERE amount > 100 (batch of 8)

Batch positions:	0 1 2 3 4 5 6 7
amount column:	20 150 5 130 200 10 90 180
predicate >100:	F T F T T F F T
Selection vector:	[1, 3, 4, 7]
Bitmap:	0 1 0 1 1 0 0 1

Next operator (e.g., compute tax = amount*1.2) can either:

- Selective: compute only for positions in SV
- Full: compute for all 0..7, then keep results where BM=1

Two ways to write the same keep/ignore note

Both options mean the same thing (“keep 1,3,4,7”), they’re just stored differently.

1) Selection Vector (SV) = list of good positions

- $SV = [1, 3, 4, 7]$

How to read it:

- “Only look at rows number 1, then 3, then 4, then 7.”

Analogy:

- a list of seat numbers of people who passed security.

2) Bitmap (BM) = 0/1 flags for each position

- $BM = 0\ 1\ 0\ 1\ 1\ 0\ 0\ 1$

How to read it:

- “At each position: 1 means keep it, 0 means ignore it.”

Analogy:

- light switches: ON = keep, OFF = ignore.

Real example: one query, step-by-step (what a DB is doing)

Imagine a normal analytics query:

```
``sql SELECT country, SUM(amount * 1.2) AS taxed_revenue FROM orders
WHERE amount > 100 GROUP BY country; ``
```

What happens (conceptually):

1) **Scan**: read columns in batches (amount[], country[]). 2) **Filter**: apply $\text{amount} > 100 \rightarrow$ update the keep/ignore note (SV or BM). 3) **Compute**: calculate $\text{amount} * 1.2$ for valid rows. 4) **Aggregate**: group by country, sum values for valid rows.

Diagram: the keep/ignore note changes across steps

Filters evolve through the pipeline (the keep/ignore note changes)



Same batch vectors can stay in memory; the “keep list” just changes after each step.

That “keep list” can be a Selection Vector (list) or a Bitmap (flags).

Why SV vs BM matters (the easy performance story)

After filtering, the next step might compute something. Example: $\text{amount} * 1.2$.

There are two styles:

Style A: Compute only on survivors (often good with SV)

If only a few rows survived, don't waste time computing the rest.

Style B: Compute on everything, then ignore bad rows (often good with BM)

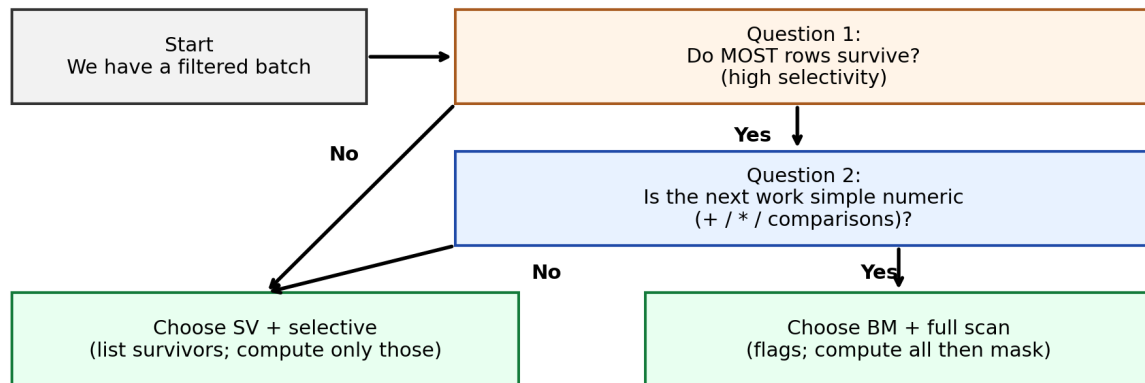
This can be faster when:

- most rows survived, and
- the work is simple math,
- the computer can do it in a very tight loop.

Cheat-sheet: what to pick (rule of thumb)

Diagram: quick decision flow

Quick decision guide (rule of thumb)



Or if you prefer a table:

If...	Usually choose...	Why
Only a few rows survive	SV	You touch only the survivors
Most rows survive and work is simple math	BM	Scanning everything can be very efficient
Work is complicated/irregular (strings, complex logic)	SV	Less overhead than dealing with many flags

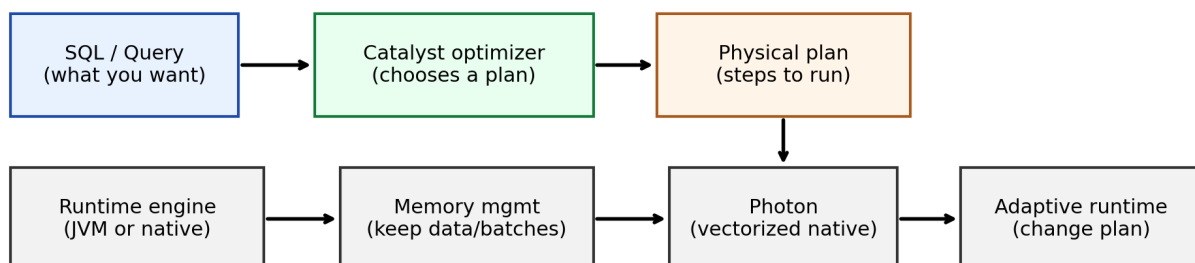
How this connects to Spark / Databricks (JVM, Catalyst, Photon, adaptivity)

These words sound scary, but they are the “bigger system” around the same theme:

Reduce wasted work, process in chunks, and use CPU + memory efficiently.

Diagram: simple view of the stack

How this relates: optimizer chooses steps, runtime executes in batches + manages memory



Memory management (simple meaning)

Memory management is how the system uses RAM so it doesn't crash and doesn't slow down.

Link to our story:

- Copying lots of data creates lots of extra memory work.
- Keeping data in place and carrying a small keep/ignore note reduces that.

JVM (simple meaning)

The JVM is the “engine” that runs Java/Scala code (Spark is mostly JVM-based).

Why it matters:

- creating lots of temporary objects can cause cleanup pauses

Catalyst query optimizer (simple meaning)

Catalyst is Spark's planner.

It decides a good way to run your query (example: push filters earlier so fewer rows survive later steps).

Photon (simple meaning)

Photon (Databricks) is a faster execution engine that is very good at batch processing and CPU-efficient loops.

Runtime adaptivity / dynamic query optimization (simple meaning)

It means:

The system can change its plan while running, based on what it learns from the data.

Example:

- It expected a filter would keep 80% of rows, but it keeps only 2%.
- It can switch strategies to match reality.

Partition coalescing (simple meaning)

A partition is a chunk of data processed in parallel.

Partition coalescing means:

If we have too many tiny chunks, merge them into fewer bigger chunks.

This is the same idea as vectorization:

- avoid doing work in tiny pieces (too much overhead)
 - do work in sensible-sized chunks
-

Conclusion

If you only remember one thing from this paper:

- Fast systems often don't "delete rows" or "copy survivors" at every step.
- They keep data in place and carry a small **keep/ignore note**.
- That note can be a **Selection Vector (list of survivors)** or a **Bitmap (0/1 flags)**.

And the deeper lesson (useful beyond databases):

Performance is often about reducing overhead and matching how the computer likes to work: in chunks, with predictable loops, and with minimal copying.